Mini Project1

Created time	October 22, 2026 10:12 AM

Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

The company currently stores transactional and analytical workloads in the same SQL database.

Tasks

1. Explain why this creates performance issues.

The company currently uses a single SQL database for both transactional processing (OLTP) and analytical processing (OLAP). This can create performance bottlenecks because these workloads have different processing requirements.

OLTP handles frequent, small operations, such as inserting customer orders, updating inventory, and processing payments. These operations require low latency and fast response times.

OLAP, on the other hand, runs complex queries involving large datasets, multiple joins, aggregations, and historical analysis. These queries consume significant CPU, memory, and disk I/O resources.

When both workloads run on the same database, analytical queries can compete with transactional operations for these shared resources, potentially increasing transaction response times and reducing overall system performance.

The main performance issues

1. Resource contention

Analytical queries may consume large amounts of CPU and memory, leaving fewer resources available for transactional operations.

Example: A sales aggregation query slows down order processing.

2. Locking and blocking

Transactions may wait when another operation holds conflicting locks on the same data. The extent of blocking depends on the database engine and isolation

level; analytical reads do not necessarily block writes.

Example: An inventory update waits for a conflicting transaction to finish.

3. Heavy disk I/O

Analytical queries can scan millions of records, generating substantial disk reads and competing with transactional queries for storage bandwidth.

Example: A historical sales report scans years of data while customers place orders.

4. Different data access patterns

OLTP typically benefits from indexes and efficient access to individual rows. OLAP often benefits from scanning and aggregating selected columns across many rows. A database optimized for one pattern may not handle the other equally efficiently.

Example: Retrieving one order is different from calculating revenue across 10 million orders.

5. Scalability and operational risk

The workloads may need different scaling strategies. Increasing resources for reporting can be costly and may still leave contention between workloads.

Example: A month-end reporting process causes slow responses during peak business hours.

Imagine an e-commerce company with 50 million order records.

- OLTP workload: The application inserts new orders, updates inventory, and records payments throughout the day.
- OLAP workload: The business intelligence team runs a query to calculate monthly revenue by product, region, and customer segment across historical order data.

If the analytical query scans a large portion of the database, it may compete with order processing for CPU, memory, storage bandwidth, and cache capacity. Depending on the database architecture, it may also create blocking or additional pressure on indexes and the transaction log.

The result could be slower order processing, longer report execution times, and increased risk of application timeouts.

2. Compare OLTP and OLAP workloads.
OLTP (Online Transaction Processing) supports day-to-day business operations, while OLAP (Online Analytical Processing) supports reporting, data analysis, and business decision-making.

Feature	OLTP	OLAP
Purpose	Run daily business transactions	Analyze business data
Examples	Placing orders, processing payments, updating inventory	Sales reports, revenue trends, customer behavior analysis
Query type	Simple, short queries	Complex queries with joins and aggregations
Data volume per query	Usually a small number of records	Often millions of records
Operations	Frequent inserts, updates, and deletes	Mostly large reads and aggregations
Response time	Milliseconds to low seconds, depending on the operation	Seconds to minutes or longer, depending on complexity
Users	Customers, applications, operational staff	Analysts, data scientists, business managers
Data model	Often normalized to reduce redundancy	Often denormalized, using star or snowflake schemas
Storage	Commonly row-oriented	Columnar storage is common
Optimization goal	High transaction throughput and low latency	Fast scans, aggregations, and analytical queries
Data scope	Current operational records	Current and historical data, often integrated from multiple sources
Scaling needs	Handle high transaction volume and concurrency	Handle large datasets and compute-intensive queries

3. Suggest an improved architecture.

1. OLTP — Transactional database

Examples: PostgreSQL, MySQL, Microsoft SQL Server

Stores orders, customers, payments, and inventory.

2. Data ingestion / CDC

Examples: Debezium + Apache Kafka, AWS DMS, Azure Data Factory

Captures new and updated records and transfers them downstream.

3. Data lake / staging (optional)

Examples: Amazon S3, Azure Data Lake Storage, Google Cloud Storage

Stores raw data and historical extracts before transformation.

4. OLAP — Analytical data warehouse

Examples: Snowflake, Google BigQuery, Amazon Redshift, Microsoft Fabric Warehouse

Stores integrated, transformed data for large analytical queries.

5. BI / reporting

Examples: Power BI, Tableau, Looker

Provides dashboards, sales reports, and business KPIs.

4. Explain where:

- **PostgreSQL /SQLServer/MySql**
- **Data warehouse**
- **Data lake**

should be used.

1. PostgreSQL / SQL Server / MySQL — OLTP databases
These are relational database management systems commonly used to run operational applications and process day-to-day business transactions.

Use them for:

- Creating and updating customer records.

- Processing orders and payments.
- Managing product inventory.
- Maintaining transactional consistency and data integrity.

Example: An e-commerce website uses PostgreSQL to save a customer's order and update inventory when a purchase is completed.

2. Data warehouse — OLAP and business analytics

A data warehouse stores integrated, structured, and curated data optimized for analytical queries and reporting.

Use it for:

- Business intelligence dashboards.
- Revenue and profitability analysis.
- Historical sales reporting.
- Aggregations across millions of records.
- Calculating KPIs across multiple business systems.

Examples of technologies: Snowflake, Google BigQuery, Amazon Redshift, and Microsoft Fabric Warehouse.

Example: Management wants to calculate monthly revenue across 10 million orders, compare sales between regions, and identify the best-selling products. The analytical query runs in the data warehouse instead of competing with live order processing.

3. Data lake — Large-scale raw and diverse data storage

A data lake stores large volumes of data in its original or processed form, including structured, semi-structured, and unstructured data.

Use it for:

- Retaining raw data from source systems.
- Storing CSV, JSON, Parquet, log files, images, and event data.
- Historical data retention and reprocessing.
- Data engineering transformations.

- Data science and machine learning workloads.

Examples of technologies: Amazon S3, Azure Data Lake Storage (ADLS), and Google Cloud Storage (GCS).

Example: An e-commerce company stores raw order events, website clickstream logs, product images, and historical customer activity in Amazon S3 for later processing and analysis.

Note that object storage such as S3 is the storage foundation, not a full analytical database by itself. Query engines and processing tools are used to analyze its contents.

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of users.

Tasks

1. Explain why BASE systems are preferred for clickstream systems.

QuickCart should use BASE-oriented distributed architecture for clickstream ingestion because it needs to handle massive event volumes, scale horizontally, and remain available during node failures. Clickstream analytics can usually tolerate eventual consistency, because a short delay before events appear in dashboards does not affect the correctness of a purchase.

BASE stands for:

- **Basically Available**: The system aims to remain available even when some components fail.
- **Soft State**: Data replicas may temporarily differ as updates propagate.
- **Eventual Consistency**: Replicas are expected to converge when updates stop and replication succeeds.

Why BASE suits clickstream data

1. High-volume ingestion

QuickCart may receive millions of clicks per minute. Distributed systems can spread events across partitions and nodes to handle high throughput.

2. Horizontal scalability

More brokers, partitions, or storage nodes can be added as event volume grows, depending on the platform.

3. Availability and fault tolerance

Replication and distributed ingestion can help the system continue accepting events when a node fails, provided the system is configured appropriately.

4. Eventual consistency is acceptable

A click may take a few seconds to appear in an analytics dashboard. That delay is usually acceptable because a single click does not need to be immediately consistent across every replica.

2. Which workloads require ACID compliance?

Workloads that require ACID compliance are those where **data accuracy, consistency, and reliable transactions are critical**.

Examples:

- **Banking transactions**: Money transfers must not lose or duplicate money.
- **Payment processing**: Payments must be recorded correctly without accidental duplicate charges.
- **Order processing**: Orders must be created reliably without duplication.
- **Inventory management**: Stock must not be oversold when customers buy simultaneously.
- **Booking systems**: Flights or hotel rooms must not be double-booked.
- **Payroll systems**: Employee salaries must be calculated and paid accurately.

3. Give examples where eventual consistency is acceptable.

Eventual consistency is acceptable when temporary delays in data updates do not affect business operations.

Examples:

- **Social media likes**: Like counts may take a few seconds to update.
- **Clickstream analytics**: Website click events can arrive late in dashboards.
- **Product recommendations**: Recommendations can use slightly outdated customer activity.

- **Search indexes**: Newly added products may take a short time to appear in search results.
- **Analytics dashboards**: Sales and traffic metrics can refresh every few minutes.
- **View counts**: Product or video views do not need to update instantly.

Q3. Storage Formats & Querying

Tasks

1. Compare:

-CSV

-JSON

-Avro

-Parquet

Feature	CSV	JSON	Avro	Parquet
Format type	Text, tabular	Text, semi-structured	Binary, row-based	Binary, column-based
Schema	No built-in schema	Flexible; optional external schema	Explicit schema	Embedded schema
Human-readable	Yes	Yes	No, generally	No, generally
Nested data	Poor support	Excellent	Supported	Supported
Compression	Supported externally	Supported externally	Efficient	Efficient
Query performance	Basic	Often slower for analytics	Good for row-based processing	Excellent for analytical queries
Common use	Simple data exchange	APIs and event data	Streaming and data pipelines	Data lakes and warehouses

2. Why do analytical systems prefer columnar storage?

Analytical systems prefer columnar storage because it reduces the amount of data read, improves compression, and speeds up aggregations across large

datasets. Formats like Parquet are therefore well suited for data warehouses and data lakes, where queries typically scan selected columns across many rows.

1. Less data scanned: A query that needs only `price` can read that column without reading `name`, `category`, or other columns.
2. Better compression: Similar values are stored together, making compression more effective and reducing storage costs.
3. Faster aggregations: Operations such as `SUM`, `AVG`, and `COUNT` can process relevant columns efficiently.

Example

Suppose a table contains 10 million sales records with 10 columns, but your query only needs `revenue`.

```
SELECT SUM(revenue)
FROM sales;
```

- Row-based storage: May need to read entire records, including columns that aren't needed.
- Columnar storage (e.g., Parquet): Can read primarily the `revenue` column, reducing I/O.

3. Explain avro vs json format.

JSON is human-readable and flexible, so it is widely used in APIs and application communication. Avro is a schema-based binary format that is generally more compact and supports schema evolution, making it useful for high-volume streaming systems such as Kafka. JSON prioritizes readability and interoperability, while Avro is often preferred for efficient, schema-aware data pipelines.

Avro: When data pipelines process large volumes of events and benefit from compact encoding, explicit schemas, and controlled schema evolution.

In Avro, the schema is often written in JSON, but the actual data is encoded in binary format.

JSON: When applications need to exchange data easily through REST APIs or when developers need to inspect events.

Feature	JSON	Avro
Format	Text-based	Binary
Human-readable	Yes	No, generally
Schema	Optional	Explicit schema
Data size	Often larger	Often smaller
Serialization	Usually more verbose	Compact and efficient
Schema evolution	Managed by application conventions or validation tools	Supports schema evolution with compatibility rules
Best use case	APIs and application data	Kafka events and data pipelines

*Serialization is about converting data into a transferable or storable format. It does not necessarily mean compressing or encrypting the data.

4. Explain compression benefits in Parquet.

Parquet uses columnar storage, which groups similar values together and makes compression more effective. This reduces storage requirements and the amount of data read during analytical queries, improving performance and potentially lowering cloud query costs.

1. **Less storage**: Compressed files take up less space in a data lake.
2. **Faster queries**: Less data needs to be read from disk or cloud storage.
3. **Lower costs**: Query engines such as Amazon Athena may charge based on the amount of data scanned, so smaller data scans can reduce costs.

Parquet supports compression codecs such as Snappy, Gzip, and Zstandard (ZSTD).

Q4. Partitioning, Indexing

Tasks

1. Design a partitioning strategy for:

-orders table

Recommended strategy: range partitioning by order date, usually monthly for a high-volume system.

Example:

Partition	Data
`orders_2026_01`	January 2026 orders
`orders_2026_02`	February 2026 orders
`orders_2026_03`	March 2026 orders

Example query:

```
SELECT *
FROM orders
WHERE order_date >= '2026-03-01'
  AND order_date <  '2026-04-01';
```

The database can skip partitions outside March. This is called partition pruning.

Benefits:

- Faster date-based queries when partition pruning applies.
- Easier archiving and deleting of old data.
- More manageable maintenance for large tables.

-clickstream data

Clickstream data records events such as product clicks, page views, searches, and add-to-cart actions. It can grow extremely quickly.

Recommended strategy: partition by event date, often daily, with optional subpartitioning or file organization by a frequently filtered field such as `event_type`.

clickstream/
├── event_date=2026-10-21/
├── event_date=2026-10-22/
└── event_date=2026-10-23/

SELECT product_id, COUNT(*) AS views
FROM clickstream
WHERE event_date = '2026-10-23'
AND event_type = 'product_view'
GROUP BY product_id;

Avoid partitioning directly by high-cardinality values such as `customer_id` in a data lake, because it can create huge numbers of small files or partitions.

2. Suggest indexing strategy for:
-customer lookups

Find a customer quickly using their customer ID or email.

Example table: `customers`

customer_id	customer_name	email
101	Alice	alice@example.com
102	Bob	bob@example.com

Indexing strategy:

- Primary key index on `customer_id`
- Unique index on `email`, if emails must be unique

```
ALTER TABLE customers
ADD PRIMARY KEY (customer_id);#if customer_id is not PK

CREATE UNIQUE INDEX idx_customers_email
ON customers (email);
```

Example query:

```
SELECT *
FROM customers
WHERE customer_id = 101;
```

The index helps the database find the customer without scanning the entire table.

-order filtering

Find orders for a particular customer within a date range.

Example table: `orders`

order_id	customer_id	order_date	total_amount
1001	101	2026-10-05	120
1002	102	2026-10-12	80
1003	101	2026-10-20	200

Indexing strategy: Create a composite index on `(customer_id, order_date)`.

```
CREATE INDEX idx_orders_customer_date
ON orders (customer_id, order_date);
```

Example query:

```
SELECT *
FROM orders
WHERE customer_id = 101
  AND order_date >= '2026-10-01'
  AND order_date < '2026-11-01';
```

The index supports filtering by customer first, then by date range.

-joins

Combine customer information with their orders.

Example tables:

- `customers.customer_id`
- `orders.customer_id`

Indexing strategy: Index the frequently used join columns. The primary key on `customers.customer_id` and an index on `orders.customer_id` are a common design.

```
CREATE INDEX idx_orders_customer_id
ON orders (customer_id);
```

Example query:

```
SELECT c.customer_name, o.order_id
FROM customers c
```

```
JOIN orders o
  ON c.customer_id = o.customer_id;
```

The index can help the database find matching orders efficiently, depending on the join plan and number of rows returned.

3. Explain when indexes negatively affect performance

Indexes can negatively affect performance because they consume storage and must be maintained during inserts, updates, and deletes. Too many or duplicate indexes increase write overhead, while indexes on low-selectivity columns may not improve read performance. Indexes that don't match query patterns may also be ineffective. Choosing indexes based on workload and validating them using execution plans and index-usage statistics is better.

ex: **1. Too many indexes**
2. Indexes on low-selectivity columns Example: `order_status` contains only `pending`, `shipped`, and `delivered`.
3. Unnecessary or duplicate indexes

```
CREATE INDEX idx_customer
ON orders (customer_id);

CREATE INDEX idx_customer_date
ON orders (customer_id, order_date);
```

4.Indexes on frequently updated columns

5. Indexes that don't match query patterns

Section B - SQL Engineering

Q3. SQL Optimization Scenario

The following query takes 18 minutes:

SELECT c.customer_name,

SUM(o.quantity * o.unit_price)

FROM orders o

```
JOIN customers c
ON o.customer_id = c.customer_id
WHERE order_date >= '2025-01-01'
GROUP BY c.customer_name;
```

Tasks

1. Identify possible bottlenecks.

1. Possible bottlenecks:

1. Full table scan: MySQL may scan all rows in `orders` to find orders from 2025 onward if no suitable index exists.
2. Join performance: If the join columns are not properly indexed, matching orders with customers may be expensive.
3. Aggregation: `SUM()` and `GROUP BY` require MySQL to calculate revenue for each customer. A large dataset may require temporary storage or sorting.
4. Incorrect grouping: Grouping only by `customer_name` may combine different customers who share the same name.
5. Data types: The `order_date` column is currently `TEXT`. Storing dates as `DATE` makes date comparisons more appropriate and allows a normal full-column index

2. Suggest indexes.

```
SELECT
c.customer_id,
c.customer_name,
SUM(o.quantity * o.unit_price) AS total_revenue
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
WHERE o.order_date >= '2025-01-01'
GROUP BY c.customer_id, c.customer_name;
```

Adding `customer_id` to the grouping prevents customers with identical names from being combined.

CREATE INDEX idx_orders_order_date
ON orders(order_date);

consider an index on the join column:

CREATE INDEX idx_orders_customer_id
ON orders(customer_id);

3. Explain execution plan analysis

Use `EXPLAIN ANALYZE` to see how MySQL actually executes the query:

```
EXPLAIN ANALYZE
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.quantity * o.unit_price) AS total_revenue
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
WHERE o.order_date >= '2025-01-01'
GROUP BY c.customer_id, c.customer_name;
```

`EXPLAIN ANALYZE` executes the query and reports actual timing and row counts, unlike a plain `EXPLAIN`, which primarily describes the optimizer's planned execution.

- - SHARED TABLES

CREATE TABLE customers (
customer_id BIGINT AUTO_INCREMENT PRIMARY KEY,
full_name VARCHAR(150) NOT NULL,
email VARCHAR(255) NOT NULL UNIQUE,
phone VARCHAR(30)
);

CREATE TABLE categories (
category_id BIGINT AUTO_INCREMENT PRIMARY KEY,

```
category_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE products (
product_id BIGINT AUTO_INCREMENT PRIMARY KEY,
category_id BIGINT NOT NULL,
sku VARCHAR(80) NOT NULL UNIQUE,
product_name VARCHAR(200) NOT NULL,
unit_price DECIMAL(12,2) NOT NULL,
CHECK (unit_price >= 0),
FOREIGN KEY (category_id)
REFERENCES categories(category_id)
);

CREATE TABLE orders (
order_id BIGINT AUTO_INCREMENT PRIMARY KEY,
customer_id BIGINT NOT NULL,
order_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
status VARCHAR(30) NOT NULL,
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
order_item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
order_id BIGINT NOT NULL,
product_id BIGINT NOT NULL,
quantity INT NOT NULL,
unit_price DECIMAL(12,2) NOT NULL,
CHECK (quantity > 0),
CHECK (unit_price >= 0),
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id),
UNIQUE KEY uq_order_item (order_id, order_item_id)
);
```

- - SHIPMENT TRACKING

```
CREATE TABLE carriers (
carrier_id BIGINT AUTO_INCREMENT PRIMARY KEY,
carrier_name VARCHAR(150) NOT NULL,
contact_phone VARCHAR(30)
);

CREATE TABLE shipments (
shipment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
order_id BIGINT NOT NULL,
carrier_id BIGINT NOT NULL,
tracking_number VARCHAR(120) NOT NULL,
status VARCHAR(30) NOT NULL DEFAULT 'pending',
shipped_at TIMESTAMP NULL,
estimated_delivery_at TIMESTAMP NULL,
delivered_at TIMESTAMP NULL,
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (carrier_id) REFERENCES carriers(carrier_id),
UNIQUE KEY uq_carrier_tracking
(carrier_id, tracking_number),
UNIQUE KEY uq_shipment_order
(shipment_id, order_id)
);

CREATE TABLE shipment_items (
shipment_item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
shipment_id BIGINT NOT NULL,
order_id BIGINT NOT NULL,
order_item_id BIGINT NOT NULL,
quantity INT NOT NULL,
CHECK (quantity > 0),
FOREIGN KEY (shipment_id, order_id)
REFERENCES shipments(shipment_id, order_id),
FOREIGN KEY (order_id, order_item_id)
REFERENCES order_items(order_id, order_item_id),
UNIQUE KEY uq_shipment_item
(shipment_id, order_item_id)
);
```

```
CREATE TABLE tracking_events (
event_id BIGINT AUTO_INCREMENT PRIMARY KEY,
shipment_id BIGINT NOT NULL,
event_status VARCHAR(40) NOT NULL,
location VARCHAR(255),
event_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
description TEXT,
FOREIGN KEY (shipment_id)
REFERENCES shipments(shipment_id)
);
```

- - PRODUCT INVENTORY

```
CREATE TABLE warehouses (
warehouse_id BIGINT AUTO_INCREMENT PRIMARY KEY,
warehouse_name VARCHAR(150) NOT NULL,
location VARCHAR(255)
);
```

```
CREATE TABLE inventory (
inventory_id BIGINT AUTO_INCREMENT PRIMARY KEY,
product_id BIGINT NOT NULL,
warehouse_id BIGINT NOT NULL,
quantity_on_hand INT NOT NULL DEFAULT 0,
quantity_reserved INT NOT NULL DEFAULT 0,
CHECK (quantity_on_hand >= 0),
CHECK (quantity_reserved >= 0),
CHECK (quantity_reserved <= quantity_on_hand),
FOREIGN KEY (product_id)
REFERENCES products(product_id),
FOREIGN KEY (warehouse_id)
REFERENCES warehouses(warehouse_id),
UNIQUE KEY uq_product_warehouse
(product_id, warehouse_id)
);
```

```
CREATE TABLE stock_movements (
movement_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
product_id BIGINT NOT NULL,
warehouse_id BIGINT NOT NULL,
movement_type ENUM(
'receipt', 'sale', 'return',
'adjustment', 'transfer'
) NOT NULL,
quantity_change INT NOT NULL,
movement_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CHECK (quantity_change <> 0),
FOREIGN KEY (product_id, warehouse_id)
REFERENCES inventory(product_id, warehouse_id)
);
```

- - CUSTOMER SUPPORT

```
CREATE TABLE support_agents (
agent_id BIGINT AUTO_INCREMENT PRIMARY KEY,
full_name VARCHAR(150) NOT NULL,
email VARCHAR(255) NOT NULL UNIQUE
);
```

```
CREATE TABLE support_tickets (
ticket_id BIGINT AUTO_INCREMENT PRIMARY KEY,
customer_id BIGINT NOT NULL,
agent_id BIGINT NULL,
order_id BIGINT NULL,
subject VARCHAR(255) NOT NULL,
status ENUM('open','pending','resolved','closed')
NOT NULL DEFAULT 'open',
priority ENUM('low','medium','high','urgent')
NOT NULL DEFAULT 'medium',
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
resolved_at TIMESTAMP NULL,
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id),
FOREIGN KEY (agent_id)
REFERENCES support_agents(agent_id),
```

```
FOREIGN KEY (order_id)
REFERENCES orders(order_id)
);

CREATE TABLE ticket_messages (
message_id BIGINT AUTO_INCREMENT PRIMARY KEY,
ticket_id BIGINT NOT NULL,
customer_id BIGINT NULL,
agent_id BIGINT NULL,
message_body TEXT NOT NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (ticket_id)
REFERENCES support_tickets(ticket_id),
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id),
FOREIGN KEY (agent_id)
REFERENCES support_agents(agent_id),
CHECK (
(customer_id IS NOT NULL AND agent_id IS NULL)
OR
(customer_id IS NULL AND agent_id IS NOT NULL)
)
);
```

Section D - ETL/ELT + Airflow

Q1. ETL vs ELT Architecture

Tasks

1. Compare ETL vs ELT.

Feature	ETL	ELT
Full form	Extract, Transform, Load	Extract, Load, Transform
Transformation	Before loading	After loading
Processing location	External processing system	Data warehouse

Feature	ETL	ELT
Raw data	May not be retained in warehouse	Usually retained
Flexibility	Less flexible for reprocessing	Easier to reprocess
Common use	Legacy systems, strict pre-load requirements	Modern cloud analytics
Example tools	Informatica, Talend	Snowflake + dbt, BigQuery + SQL

2. Explain why cloud warehouses prefer ELT.

Modern cloud warehouses such as Snowflake, Google BigQuery, and Amazon Redshift are well suited to ELT because:

1. Scalable computing: They can process large datasets using distributed computing.
2. Faster ingestion: Raw data can be loaded without waiting for all transformations.
3. Flexible transformations: Data can be cleaned and transformed later using SQL.
4. Raw data preservation: Original data can be retained for auditing and reprocessing.
5. Reduced infrastructure management: Many transformations can run directly in the warehouse instead of requiring a separate processing server.

3. Design modern ELT pipeline for QuickCart.

QuickCart should use a modern ELT architecture because it supports scalable cloud-based data processing.

In ETL, data is extracted, transformed, and then loaded into a warehouse. In ELT, data is extracted and loaded into the warehouse first, and transformations are performed inside the warehouse.

Cloud warehouses prefer ELT because they offer scalable processing, flexible SQL transformations, faster raw-data ingestion, and the ability to retain original data for auditing and reprocessing.

For QuickCart, customer, order, product, and clickstream datasets can be extracted using Python and loaded into a cloud data warehouse such as Snowflake. SQL or dbt can then clean and transform the data into analytics-ready tables. Apache Airflow can orchestrate the pipeline by scheduling ingestion, validation, transformation, and reporting tasks. The final processed data can be used in Power BI or Tableau to analyze sales, customer activity, and website traffic.

QuickCart has four datasets from your mini-project:

- `customers.csv`
- `orders.csv`
- `products.csv`
- `clickstream.csv`

A modern ELT architecture could look like this:

ELT Pipeline Flow
Data Sources Customers · Orders · Products · Clickstream
↓
Ingestion Python extracts and uploads CSV files
↓
Cloud Warehouse — Raw Layer raw_customers · raw_orders · raw_products · raw_clickstream
↓
SQL / dbt Transformations Validate · Deduplicate · Standardize · Join
↓
Analytics Layer Clean customers · Sales summaries · Clickstream sessions
↓
Dashboards & Reports Sales performance · Customer behavior · Traffic analytics

ELT Pipeline Flow
Apache Airflow orchestrates the pipeline Airflow schedules tasks, manages dependencies, retries failures, and monitors execution.